

# Algoritmos Avanzados



## Capítulo 1 – Estrategias Algorítmicas Programación Dinámica

**Profesores:**

**Tupia, M.**

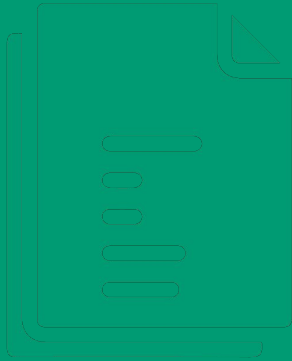
0681 - 0682

**Cueva, R.**

0683

**Corcuera, J.**

0684



## Contenidos

- Resultados de aprendizaje
- Definiciones
- Ejemplos Clásicos de Programación Dinámica
- Conclusiones
- **Bibliografía**

# Resultados de aprendizaje

Primera parte



## Resultados de aprendizaje

- RA3: Comunica la solución a problemas de ingeniería mediante una propuesta oral y escrita, aplicando las estrategias algorítmicas desarrolladas en el curso y validando su eficiencia mediante la experimentación de sus resultados

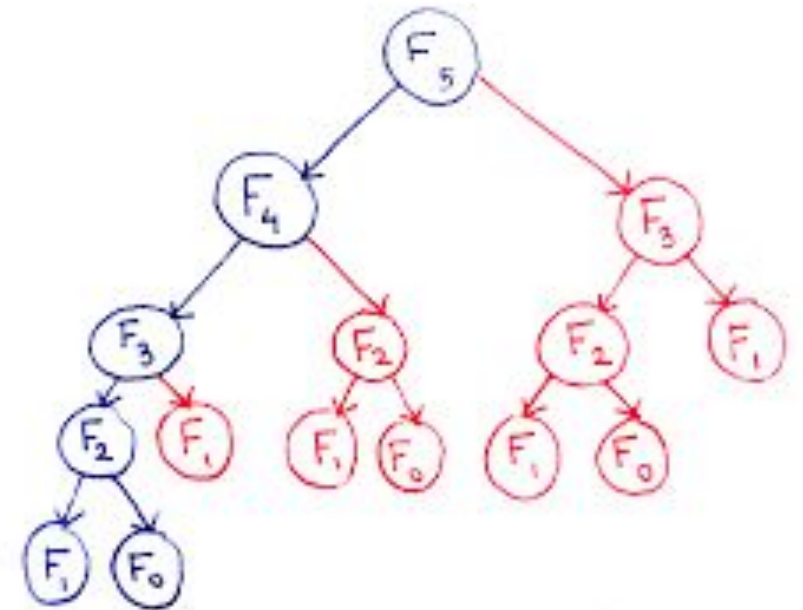


# Definiciones

Segunda parte

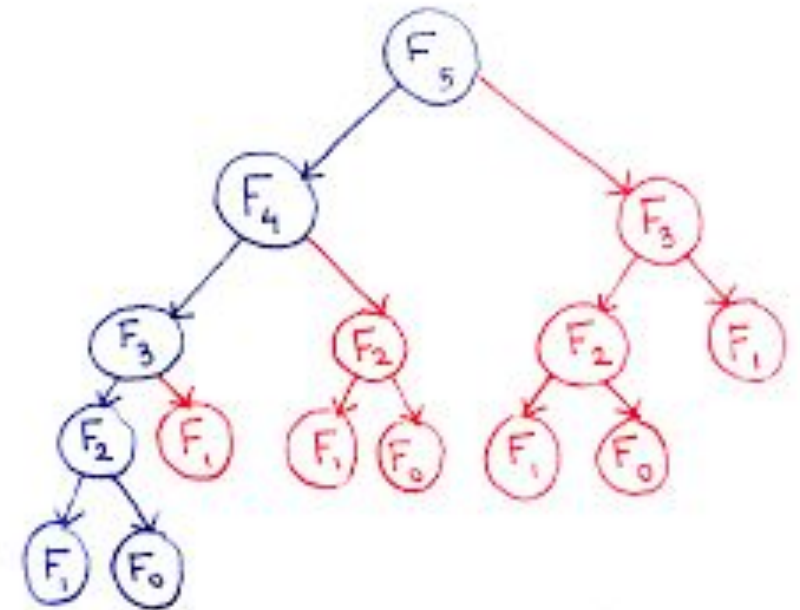
# Programación Dinámica

- Es una técnica de diseño de algoritmos inventado por Richard Bellman (1950) como un método general para optimizar los procesos de decisión de múltiples etapas.
- Por lo tanto, la palabra "programación" en el nombre de esta técnica significa "Planificación" y no se refiere a la codificación de programas de computadoras necesariamente.
- Después de probar su valor como una herramienta importante de las matemáticas aplicadas, la programación dinámica llega a ser considerada, como una técnica de diseño de algoritmos que no tiene que limitarse a tipos especiales de problemas de optimización.



# Programación Dinámica

- A veces, la forma natural de dividir una instancia sugerida por la estructura del problema nos lleva a considerar varias subinstancias superpuestas
- Si se resuelve cada uno de estos en forma independiente, a su vez crearán un gran número de subinstancias idénticas
- Si no se presta atención a esta duplicación, es probable que se termine con un algoritmo ineficiente
- Si, por otro lado, se aprovecha de la duplicación y se resuelve cada subinstancia sólo una vez, salvando la solución para su uso posterior, resultará en un algoritmo más eficiente

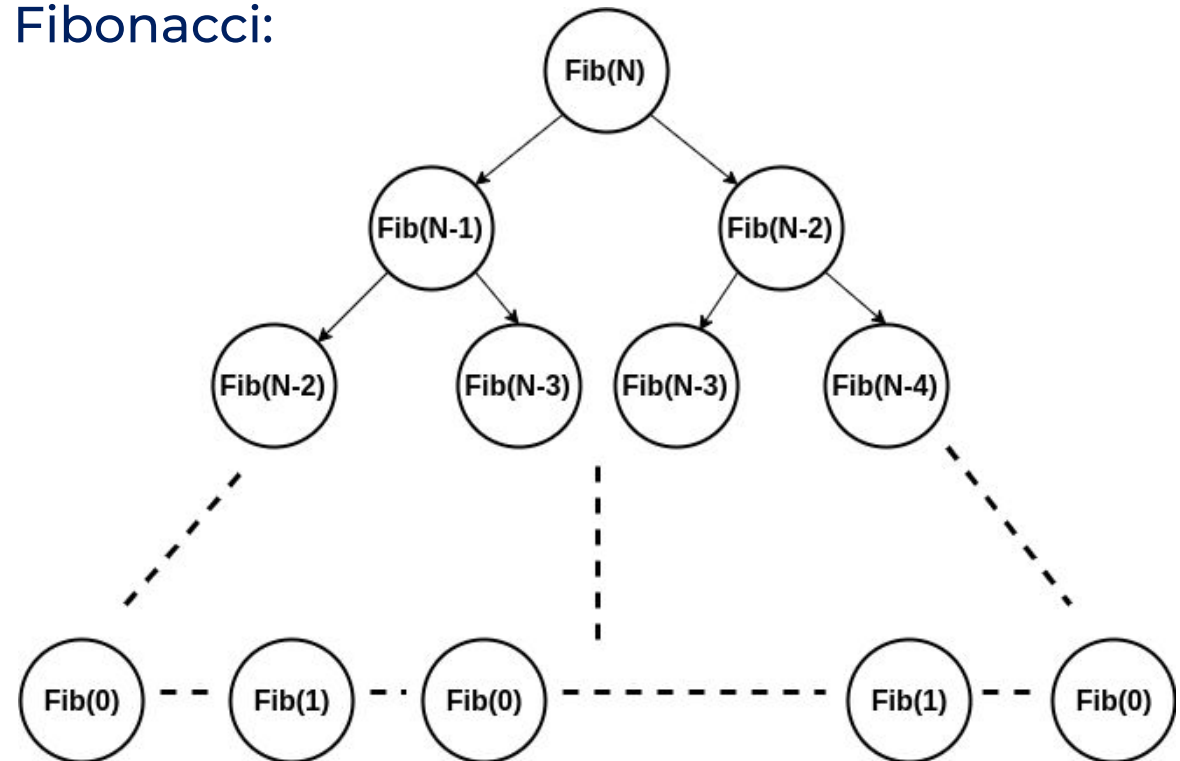


## Ejemplo 1: Números de Fibonacci

❖ Recordar la definición de los números de Fibonacci:

- $F(n) = F(n-1) + F(n-2)$
- $F(0) = 0$
- $F(1) = 1$

❖ Calcular el n-ésimo número de Fibonacci recursivamente (top-down)





## Ejemplo 1: Números de Fibonacci

- ❖ Calcular el  $n$ -ésimo número de Fibonacci usando iteraciones bottom-up y almacenando los resultados:

- $F(0) = 0$

- $F(1) = 1$

- $F(2) = 1 + 0 = 1$

- ...

- $F(n-2) =$

- $F(n-1) =$

- $F(n) = F(n-1) + F(n-2)$

|   |   |   |       |          |          |        |
|---|---|---|-------|----------|----------|--------|
| 0 | 1 | 1 | . . . | $F(n-2)$ | $F(n-1)$ | $F(n)$ |
|---|---|---|-------|----------|----------|--------|

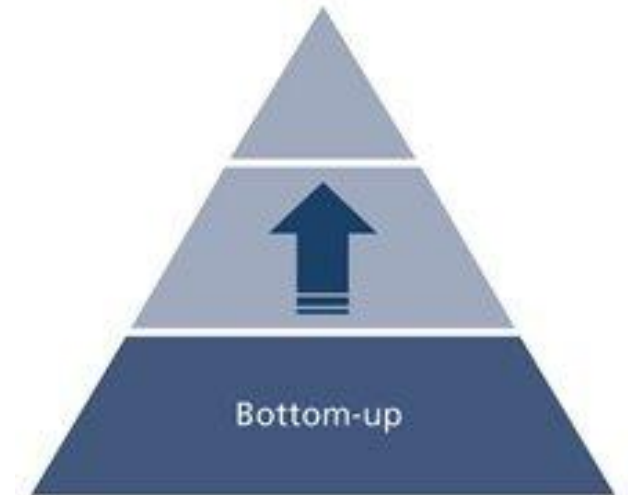
## Programación Dinámica

- ❖ La idea subyacente es evitar el cálculo de una misma instancia dos veces
- ❖ Normalmente, se usa un tabla de resultados conocidos, que se llenan con las subinstancias resueltas.

| <i>m</i> | <i>i</i> ↓ | <i>Valor j</i> → |   |   |   |   |   |   |   |   |
|----------|------------|------------------|---|---|---|---|---|---|---|---|
|          |            | 0                | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 0        | 0          | 0                | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1        | 1          | 0                | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 3        | 2          | 0                | 1 | 2 | 1 | 2 | 3 | 2 | 3 | 4 |
| 5        | 3          | 0                | 1 | 2 | 1 | 2 | 1 | 2 | 3 | 2 |
| 6        | 4          | 0                | 1 | 2 | 1 | 2 | 1 | 1 | 2 | 2 |

## Programación Dinámica

- ❖ La programación dinámica es una técnica bottom-up (de abajo hacia arriba)
  - Se comienza con la subinstancia más pequeña:
    - La “más simple”
  - Mediante la combinación de sus soluciones, se obtienen las respuestas a subinstancias de tamaño creciente, hasta que finalmente se llega a la solución de la instancia original.



## El principio de optimalidad

La programación dinámica se utiliza a menudo para resolver problemas de optimización que satisfacen el principio de optimalidad

- En una secuencia óptima de decisiones o elecciones, cada subsecuencia también debe ser óptima
- Si bien este principio puede parecer obvio, no siempre se aplica.

## El principio de optimalidad

El principio de optimalidad se puede definir de la siguiente manera

- La solución óptima para cualquier caso no trivial es una combinación de soluciones óptimas de algunas de sus subinstancias
- La dificultad en la transformación de este principio en un algoritmo es que no suele ser evidente cuales subinstancias son relevantes para el problema

## Ejemplo: Coin Row Problem

- Existe una fila de  $n$  monedas cuyos valores son los enteros positivos  $c_1, c_2, \dots, c_n$ , no necesariamente distintos. El objetivo es escoger la máxima cantidad de dinero considerando que no se pueden seleccionar al mismo tiempo dos monedas adyacentes en la fila inicial
- Ejemplo: 5, 1, 2, 10, 6, 2.
- ¿Cuál es la mejor selección?



## Ejemplo: Coin Row Problem

- Sea  $F(n)$  la cantidad máxima que se puede obtener con  $n$  monedas.
- Para derivar una recurrencia  $F(n)$ , dividimos todas las selecciones permitidas de monedas en dos grupos:
  - Aquellas con la última moneda
  - Aquellas sin la última moneda
- Entonces, tenemos la siguiente recurrencia:
  - $F(n) = \max\{cn + F(n-2), F(n-1)\}$  para  $n > 1$ ,
  - $F(0) = 0, F(1)=c1$

## Ejemplo: Coin Row Problem

$$F[0] = 0, F[1] = c_1 = 5$$

| index | 0 | 1 | 2 | 3 | 4  | 5 | 6 |
|-------|---|---|---|---|----|---|---|
| C     |   | 5 | 1 | 2 | 10 | 6 | 2 |
| F     | 0 | 5 |   |   |    |   |   |

$$F[2] = \max\{1 + 0, 5\} = 5$$

| index | 0 | 1 | 2 | 3 | 4  | 5 | 6 |
|-------|---|---|---|---|----|---|---|
| C     |   | 5 | 1 | 2 | 10 | 6 | 2 |
| F     | 0 | 5 | 5 |   |    |   |   |

$$F[3] = \max\{2 + 5, 5\} = 7$$

| index | 0 | 1 | 2 | 3 | 4  | 5 | 6 |
|-------|---|---|---|---|----|---|---|
| C     |   | 5 | 1 | 2 | 10 | 6 | 2 |
| F     | 0 | 5 | 5 | 7 |    |   |   |

$$F[4] = \max\{10 + 5, 7\} = 15$$

| index | 0 | 1 | 2 | 3 | 4  | 5 | 6 |
|-------|---|---|---|---|----|---|---|
| C     |   | 5 | 1 | 2 | 10 | 6 | 2 |
| F     | 0 | 5 | 5 | 7 | 15 |   |   |

$$F[5] = \max\{6 + 7, 15\} = 15$$

| index | 0 | 1 | 2 | 3 | 4  | 5  | 6 |
|-------|---|---|---|---|----|----|---|
| C     |   | 5 | 1 | 2 | 10 | 6  | 2 |
| F     | 0 | 5 | 5 | 7 | 15 | 15 |   |

$$F[6] = \max\{2 + 15, 15\} = 17$$

| index | 0 | 1 | 2 | 3 | 4  | 5  | 6         |
|-------|---|---|---|---|----|----|-----------|
| C     |   | 5 | 1 | 2 | 10 | 6  | 2         |
| F     | 0 | 5 | 5 | 7 | 15 | 15 | <b>17</b> |



## Ejemplo: Coin Row Problem

**ALGORITHM** *CoinRow*( $C[1..n]$ )

//Applies formula (8.3) bottom up to find the maximum amount of money

//that can be picked up from a coin row without picking two adjacent coins

//Input: Array  $C[1..n]$  of positive integers indicating the coin values

//Output: The maximum amount of money that can be picked up

$F[0] \leftarrow 0; \quad F[1] \leftarrow C[1]$

**for**  $i \leftarrow 2$  **to**  $n$  **do**

$F[i] \leftarrow \max(C[i] + F[i - 2], F[i - 1])$

**return**  $F[n]$

# Ejemplos Clásicos de Programación Dinámica

Tercera parte

## Longest Increasing Subsequence (LIS)

- Dada una secuencia  $\{X[0], X[1], \dots, X[n-1]\}$ , determinar la longitud de la subsecuencia más larga de tal forma que todos los elementos de la subsecuencia estén ordenados en forma creciente
  - La “subsecuencia” no es necesariamente continua

Ejemplo:

- Secuencia =  $\{-7, 10, 9, 2, 3, 8, 8, 1\}$ ,  $N = 8$
- LIS =  $\{-7, 2, 3, 8\}$ , Longitud = 4

## Longest Increasing Subsequence (LIS)

- Sea  $LIS(i)$  la LIS que termina en el índice  $i$

| Índice | 0  | 1  | 2 | 3 | 4 | 5 | 6 | 7 |
|--------|----|----|---|---|---|---|---|---|
| X      | -7 | 10 | 9 | 2 | 3 | 8 | 8 | 1 |
| LIS(i) | 1  | 2  | 2 | 2 | 3 | 4 | 4 | 2 |

- $LIS(0) = 1: \{-7\}$
- $LIS(1) = 2: \{-7, 10\}$
- $LIS(2) = 2: \{-7, 9\}$
- $LIS(3) = 2: \{-7, 2\}$
- $LIS(4) = 3: \{-7, 2, 3\}$

- $LIS(5) = 4: \{-7, 2, 3, 8\}$

- $LIS(6) = 4: \{-7, 2, 3, 8\}$

- $LIS(7) = 2: \{-7, 1\}$

## Longest Increasing Subsequence (LIS)

Sea  $L(i)$  la longitud de la LIS que termina en  $i$ :

$$L(i) = \begin{cases} 1 + \max_{j=0, \dots, i-1} \{L(j) \mid X_j < X_i\}, & i > 0 \\ 1 & i = 0 \end{cases}$$

Lo anterior permite obtener la longitud de la subsecuencia, ¿cómo se obtiene los elementos que la conforman?

- Usar una estructura que permita almacenar el índice de la subsecuencia  $L(j)$  a la que se agregó  $i$



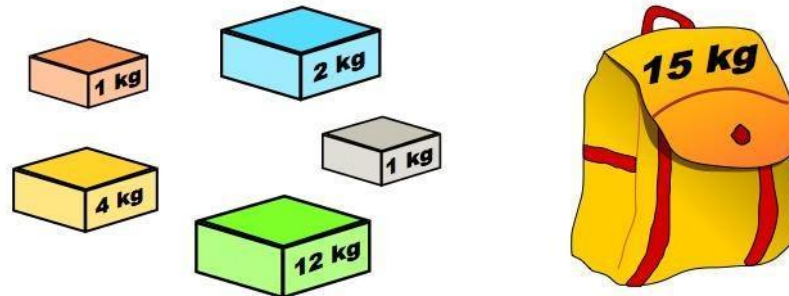
## Knapsack Problem

Dados  $n$  elementos de

- pesos enteros:  $w_1 \ w_2 \ \dots \ w_n$
- valores:  $v_1 \ v_2 \ \dots \ v_n$

y una mochila de capacidad entera  $W$ .

Encontrar el subconjunto más valioso de elementos que caben en la mochila.



## Knapsack Problem

Consideremos la instancia definida por los primeros  $i$  elementos y capacidad  $j$  ( $j \leq W$ )

Sea  $V[i,j]$  el valor óptimo de dicha instancia. Luego:

$$V[i,j] = \begin{cases} \max \{V[i-1,j], V_i + V[i-1,j - W_i]\} & , j - W_i \geq 0 \\ V[i-1,j] & , j - W_i < 0 \end{cases}$$

Condiciones iniciales:  $V[0,j] = 0$  y  $V[i,0] = 0$

## Ejemplo: Knapsack Problem

Consideremos la instancia dada por los siguientes datos:

| Elemento | Peso ( $w$ ) | Valor ( $v$ ) |
|----------|--------------|---------------|
| 1        | 2            | \$12          |
| 2        | 1            | \$10          |
| 3        | 3            | \$20          |
| 4        | 2            | \$15          |

Capacidad  $W = 5$



## Ejemplo: Knapsack Problem

$$V[i,j] = \begin{cases} \max \{V[i-1,j], v_i + V[i-1,j - w_i]\} & , j - w_i \geq 0 \\ V[i-1,j] & , j - w_i < 0 \end{cases}$$

Condiciones iniciales:  $V[0,j] = 0$  y  $V[i,0] = 0$

|               |            |   | Capacidad $j$ |    |    |    |    |           |
|---------------|------------|---|---------------|----|----|----|----|-----------|
| Elementos $i$ |            |   | 0             | 1  | 2  | 3  | 4  | 5         |
|               |            | 0 | 0             | 0  | 0  | 0  | 0  | 0         |
| $w_1 = 2$     | $v_1 = 12$ | 1 | 0             | 0  | 12 | 12 | 12 | 12        |
| $w_2 = 1$     | $v_2 = 10$ | 2 | 0             | 10 | 12 | 22 | 22 | 22        |
| $w_3 = 3$     | $v_3 = 20$ | 3 | 0             | 10 | 12 | 22 | 30 | 32        |
| $w_4 = 2$     | $v_4 = 15$ | 4 | 0             | 10 | 15 | 25 | 30 | <b>37</b> |

# Conclusiones

Quinta parte

## Conclusiones

- La **programación dinámica** es una estrategia para resolver problemas con subproblemas que se superponen
- Típicamente, estos subproblemas surgen de una **recurrencia** que relaciona la solución a un problema dado con soluciones a sus subproblemas (que son del mismo tipo)
- Se sugiere resolver cada subproblema de menor tamaño una vez y **grabar los resultados en una tabla** de la que se puede obtener la solución al problema original



## Referencias

LEVITIN, A. **Introduction to The Design and Analysis of Algorithms**. 3ra edición. USA: Pearson, 2012. ISBN 0-13-231681-1.

- Cap8: Dynamic Programming
  - 8.1 Three Basic Examples
  - 8.2 The Knapsack Problem
    - No considerar la sección de Memory Functions

HALIM, Steven and HALIM, Felix. **Competitive Programming**. 2da edición. 2011.

- Cap3: Problem Solving Paradigms
  - 3.4 Dynamic Programming